

1.1. RISC-V Processor Overview

Consider the RISC-V processor as a device under test, and our objective is to locate different fault paths inside the processor architecture, including flip-flops and channel wires. We have chosen the NEORV32 RISC-V Processor, which is an open-source RISC-V compatible processor system designed to function as a ready-to-go auxiliary processor within a larger SoC (System-on-Chip) design or as a stand-alone customisable microcontroller [36]. The central processing unit of the NEORV32 SoC is shown in Figure 1.

The primary units in the processor architecture are the CPU control unit, CPU Register file, Arithmetic Logic Unit (ALU), and CPU BUS Unit. The front-end of the CPU control unit is responsible for fetching instructions, while the back end performs the actual execution. The ALU is used for processing data from the register file, and for memory and branch address computations. The bus unit takes care of handling data memory accesses via load and store instructions [36].

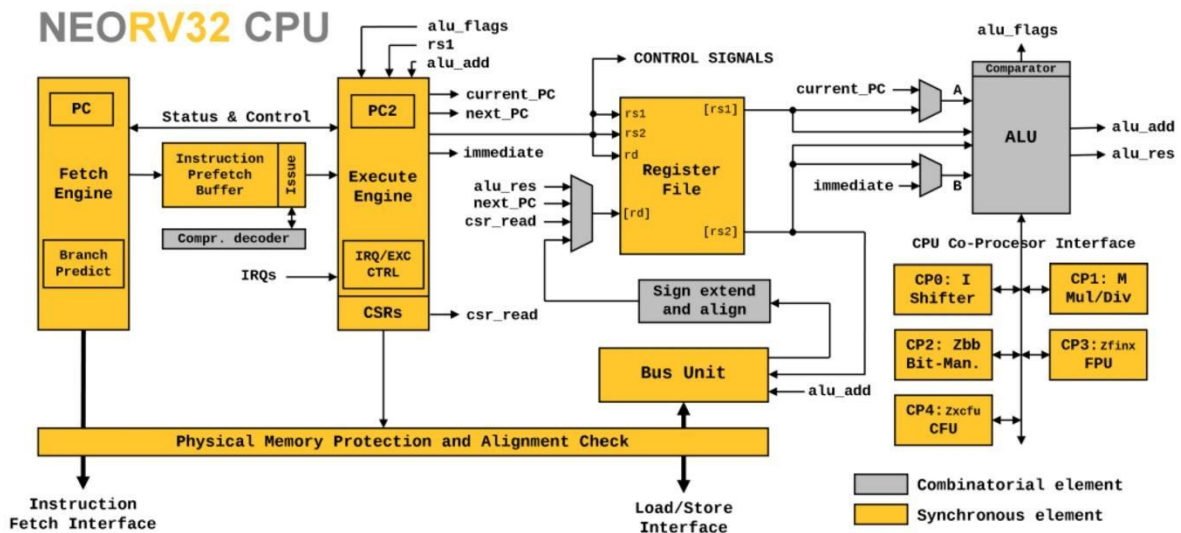


Figure 1. NEORV32 RISC-V Central Processing Unit [36]

1.2. RISC-V programming toolchain and UART interface

This section of the report discusses the RISC-V programming toolchain, which is employed to compile C code into executable binary files. It will outline the steps required to install and utilize this toolchain within a Linux Ubuntu environment. Subsequently, an overview of the current software testbench setup on the Ultra96+ FPGA board will be provided. Lastly, a brief demonstration will be presented, illustrating the process of uploading a program to the NEORV32 CPU on the FPGA using UART.

1.2.1. RISC-V programming toolchain

The RISC-V architecture employs a unique instruction set not natively supported by the standard GCC compiler. The RISC-V programming toolchain fills this gap, enabling the translation of C source code into machine code that is executable for RISC-V-based systems.

The RISC-V toolchain currently used to generate executable files for testing the NEORV32 and the fault injection module that is developing by Nidhin, is provided by the developer of the NEORV32 CPU, which works only in the Linux environment.

1.2.2. Installation

Although the programming toolchain is designed to function on any Linux platform, Ubuntu is highly recommended. This is because Ubuntu is natively equipped with all the necessary libraries required by the toolchain. From my experience, some old OS like Centos 7, which are no longer supported, may face difficulties in installing the newer version of certain libraries.

Installation itself is straightforward. The programming toolchain can be downloaded from <https://github.com/stnolting/riscv-gcc-prebuilt> [37]. After the downloading, the binaries folder (bin) should be added to the PATH environment variable by command: `cd`, where `opt` should be replaced by your path to the riscv toolchain folder. To ensure the RISC-V toolchain is available every time a new console is started, add this command to “.bashrc” file.

After the installation, validation can be done by navigating to a NEORV32 software folder (such as `neorv32/sw/example/demo_blink_led`) and execute “make check”. The console should print “toolchain check ok” if the installation was successful.

1.2.3. Usual toolchain command and installation validation

The toolchain introduces variety of commands to the Linux console. Each command begins with “make”, followed by specific targets, as illustrated in Figure 52. To compiling program for RISC-V CPU, we primarily focus on “exe”, “install”, “bootloader” and “clean_all” target.

The “make install” command will generate the application image, which contain the instructions to be executed by the CPU. This image is used by directly copy and replace the “application_image.vhd” file in the NEORV32 RTL folder, which will be automatically written into the IMEM of the CPU during the synthesis. The “make exe” command generate the binary file that can be uploaded to the NEORV32 through UART port and the bootloader. The “make bootloader” command generate a bootloader image, which can be used like the application image, except the generated code should be copy to the “BOOTROM.vhd” rather than the “application_image.vhd”. And “make clean_all” will remove everything that is generated by the toolchain, it’s often used together with previous three targets (such as “make clean_all exe”) to make sure old file are removed when compiling a new program.

```

help      - show this text
check     - check toolchain
info      - show makefile/toolchain configuration
gdb       - run GNU debugger
asm       - compile and generate <main.asm> assembly listing file for manual debugging
elf       - compile and generate <main.elf> ELF file
exe       - compile and generate <neorv32_exe.bin> executable for upload via default bootloader (binary file, with header)
bin       - compile and generate <neorv32_raw_exe.bin> RAW executable file (binary file, no header)
hex       - compile and generate <neorv32_raw_exe.hex> RAW executable file (hex char file, no header)
image     - compile and generate VHDL IMEM boot image (for application, no header) in local folder
install   - compile, generate and install VHDL IMEM boot image (for application, no header)
sim       - in-console simulation using default/simple testbench and GHDL
all       - exe + install + hex + bin + asm
elf_info  - show ELF layout info
clean     - clean up project home folder
clean_all - clean up whole project, core libraries and image generator
bl_image  - compile and generate VHDL BOOTROM boot image (for bootloader only, no header) in local folder
bootloader - compile, generate and install VHDL BOOTROM boot image (for bootloader only, no header)

```

Figure 52. Command list of the RISC-V programming toolchain

As an example, Figure 53 is a folder contains all essential file needed to compile a RISC-V program: a source file written in C, and a makefile which is provided in the NEORV32 project. After executing the “make clean_all exe” command, the output file “neorv32_exe.bin” is generated, as illustrated in Figure 54. This file can subsequently be uploaded to the NEORV32 via the bootloader.

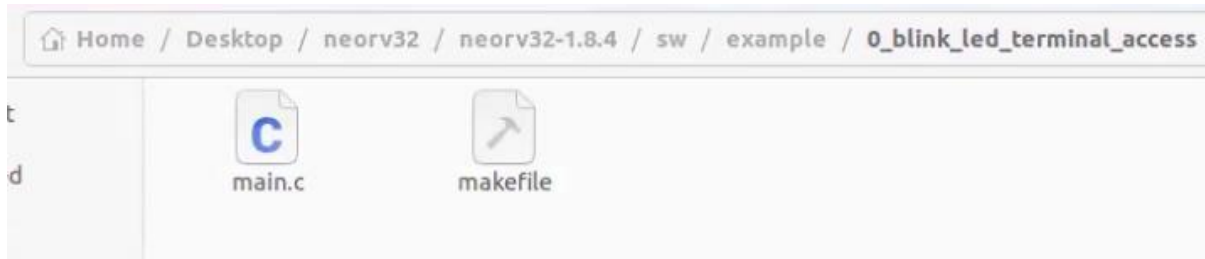


Figure 53. Folder content of an example program before compilation



Figure 54. Folder content of an example program after ran command “make clean_all exe”. “neorv32_exe.bin” is the output file we are looking for; other new files are intermediate files used for generating the output file.

1.3. UART and bootloader

In its minimum configuration, the NEORV32 automatically runs the program instructions stored in the IMEM of the CPU, which are fixed during the synthesis. Although this set up may save FPGA resource, it’s not ideal for our purpose of using NEORV32 as a testbench of a fault tolerant system. Especially when evaluating the system’s response to different programs.

The NEORV32 is equipped with bootloader and UART modules. With these components works together, program can be uploaded to the CPU for execution even after the synthesis. Furthermore, the UART module give the CPU capability to print information to a PC terminal, which allow a more complex program design, and may be helpful to make the consequences of errors and “error fixed” easier to observe and record.

1.3.1. UART setup of NEORV32

Enabling the UART module on the NEORV32 is straightforward. Firstly, set generics generic “IO_UART0_EN” and “INT_BOOTLOADER_EN” in the NEORV32 SoC top-level to true, as illustrated in Figure 55. Then, extra IO ports for UART RX and TX should be introduced into the top-level design. After that, these top-level ports can be mapped to a physical GPIO on the FPGA by constraint file as normal.

```
IO_UART0_EN           => true,
INT_BOOTLOADER_EN     => true
```

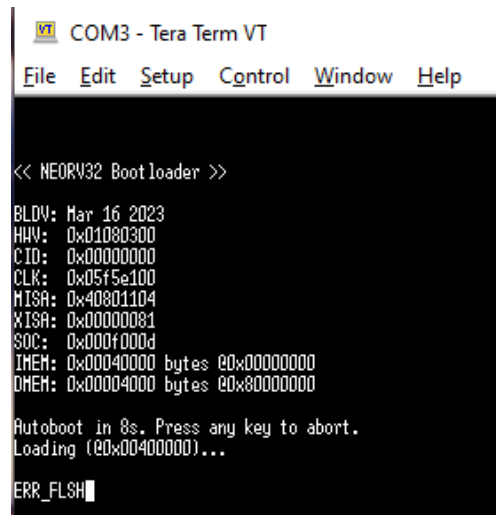
Figure 55. generics used to enable UART and bootloader modules on the NEORV32.

A UART to USB converter for connecting UART interface on CPU (RX and TX on FPGA GPIO) to PC, and a terminal program are required to establish the connection. Although there

are many terminal programs available, Tera Term is the recommended choice for our application. Primarily because its “send file” function, which is essential to transmitting the application to the NEORV32. In addition, the parameters of NEORV32’s UART module are: 19200 Baud; 8 data bits; 1 stop bit; no parity bits; no flow control protocol; receiver newline on \r\n.

1.3.2. Bootloader and program uploading demonstration.

Once the CPU been programmed on to the FPGA (with UART and bootloader module enabled) and with UART interface and connection correctly set up, the bootloader should display some basic CPU information such as memory size and clock frequency, as illustrated in Figure 56. The bootloader then waits for 8 seconds (This duration can be modified in the bootloader’s source file), before attempting to load a program from the external SPI memory. In our current setup, because the SPI module is not enabled, and there is no external SPI memory connected to the CPU, the bootloader reports an error after the 8 second period.



```

COM3 - Tera Term VT
File Edit Setup Control Window Help

<< NEORV32 Boot loader >>
BLDV: Mar 16 2023
HVV: 0x01080300
CID: 0x00000000
CLK: 0x05f5e100
MISA: 0x40801104
XISA: 0x00000081
SOC: 0x000f000d
IMEM: 0x00040000 bytes 00x00000000
OMEM: 0x00004000 bytes 00x80000000

Autoboot in 8s. Press any key to abort.
Loading (0x00400000)...

ERR_FLSH
  
```

Figure 56. Bootloader initiation information

To restart the bootloader, press the button that is connected to the “rst” port of the NEORV32’ top-level design. Subsequently, press any key within the terminal to halt the autoboot process. The bootloader will display all available commands, as depicted in Figure 57. Here is a quick guide to commands:

- Press ‘h’ will display the command list again.
- ‘r’ restarts the bootloader, presenting the basic CPU information and initiating the autoboot process from the external SPI memory as before.
- ‘u’ let the bootloader stand by and ready to receive an application.
- Commands ‘s’, ‘l’, and ‘x’ are all for interactions with the SPI module.
- ‘e’ will let the CPU start to execute the currently loaded application.



```

Available CMDs:
h: Help
r: Restart
u: Upload
s: Store to flash
l: Load from flash
x: Boot from flash (XIP)
e: Execute
CMD:> 
  
```

Figure 57. CMD list displayed by the bootloader after aborting the autoboot from SPI memory.

For our primary use of the bootloader, which is the application uploading, press ‘u’. The bootloader should start waiting to receive application, as shown in Figure 58. Next, on the Tera Term interface, click “File” -> “Send file”, a window as in Figure 59 will appear. Before proceeding to the uploading, it is crucial to check the “Binary” option as highlighted in Figure 59. After this, navigate to the folder containing an application generated with RISC-V toolchain and click open to upload it. After uploading completes, it is typically necessary to navigate to “set up” -> “general” and click OK to reset the language on the terminal. This step prevents terminal display mojibake for subsequent content. After the program uploading finished, press ‘e’ to start executing the application. Figure 60 is an example of a Fibonacci calculator. Here, pressing s will calculate next 10 fibonacci number, and press e to restart the sequence.

```
CMD:> uu
Awaiting neorv32_exe.bin...
```

Figure 58. bootloader awaiting an application.

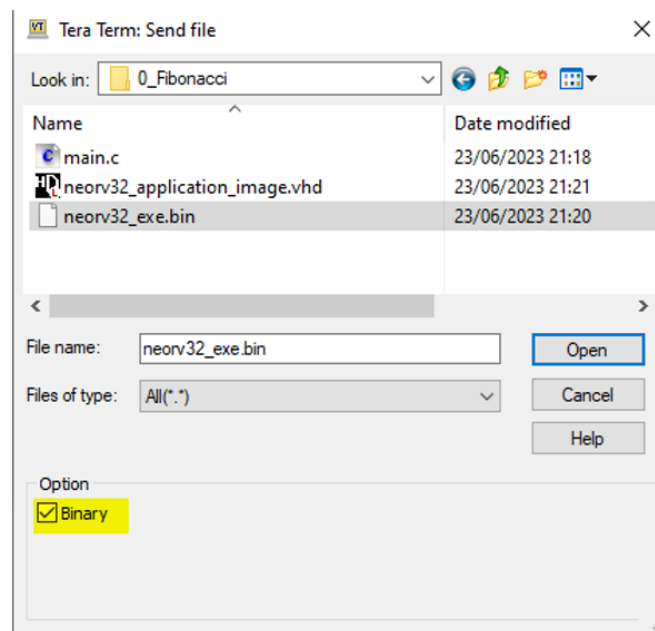


Figure 59. Tera Term’s “send file” window.

```
COM3 - Tera Term VT
File Edit Setup Control Window Help
Press 's' to calculate 10 more Fibonacci numbers or 'e' to restart sequence.
0123
456789abcdef01234567890K
CMD:> ee
Booting from 0x00000000...
Press 's' to calculate 10 more Fibonacci numbers or 'e' to restart sequence.
s0 1 1 2 3 5 8 13 21 34 55
s89 144 233 377 610 987 1597 2584 4181 6765
s10946 17711 28657 46368 75025 121393 196418 317811 514229 832040
s1346269 2178309 3524578 5702887 9227465 14930352 24157817 39088169 63245986 102
334155
e0 1 1 2 3 5 8 13 21 34 55
s89 144 233 377 610 987 1597 2584 4181 6765
```

Figure 60. Program execution

References

stnolting, "The NEORV32 RISC-V Processor: Datasheet," [Online]. Available: <https://stnolting.github.io/neorv32/>.

stnolting, "The NEORV32 RISC-V Processor: User Guide," [Online]. Available: https://stnolting.github.io/neorv32/ug/#_uploading_and_starting_of_a_binary_executable_image_via_uart. [Accessed 2023].